

Wondersale-OS: Ubuntu 24.04 Deployment Guide

Comprehensive step-by-step production deployment manual for user *jason*.

Complete Ubuntu 24.04 Production Deployment Guide for Wondersale-OS

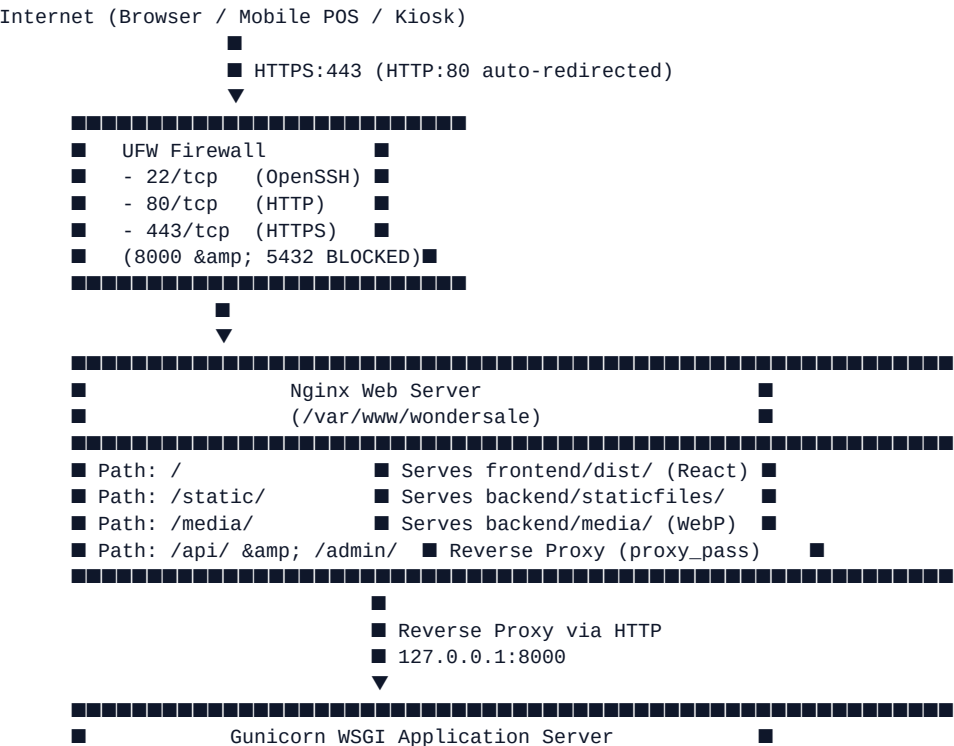
Target OS: Ubuntu 24.04 LTS (Noble Numbat)
Server Hostname: Wondersale
Application User: `jason` (non-root with `sudo`)`
Deployment Directory: ``/var/www/wondersale``
Database: PostgreSQL 16 (Local)
Python Version: Python 3.12 (via ``python3 -m venv``)
Node.js Version: Node.js 22 LTS (via NodeSource)
App Server: Gunicorn WSGI via ``systemd`` (`wondersale.service``)
Web Server & Reverse Proxy: Nginx 1.24+ with SSL via Let's Encrypt Certbot

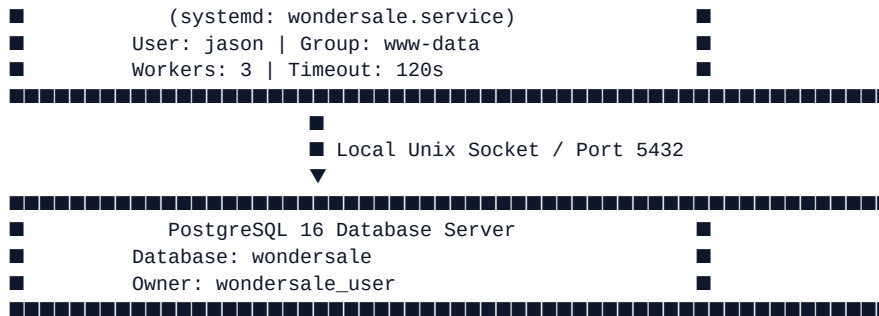
1. Overview & Architecture

Wondersale-OS is structured as a two-tier application:

- Frontend:** React 19 Single Page Application built with Vite into optimized static HTML, CSS, and JS bundles (``frontend/dist``). Nginx serves these static assets directly to client browsers at lightning speed.
- Backend:** Django 5.2 / Django REST Framework application running inside a dedicated Python 3.12 virtual environment. Gunicorn manages backend WSGI worker processes listening on local port ``127.0.0.1:8000``. Nginx reverse-proxies ``/api/``, ``/admin/``, and serves ``/static/`` and ``/media/`` files.

1.1 Architecture Diagram





1.2 Prerequisites & Secret Placeholders

Before beginning, substitute these placeholder values throughout the configuration files and commands:

| Placeholder | Meaning / Example | How to Generate or Obtain |

|---|---|---|

| `YOUR\_DOMAIN` | Your primary domain name (e.g. `wonder.sale` or `pos.example.com`) | Purchase from registrar (Namecheap, Cloudflare, GoDaddy). Create DNS A record pointing to `YOUR\_SERVER\_IP`. |

| `YOUR\_SERVER\_IP` | Public IPv4 of your VPS (e.g. `198.51.100.42`) | Provided in your VPS hosting dashboard (Hetzner, DigitalOcean, Linode, AWS). |

| `DB\_PASSWORD` | PostgreSQL password for `wondersale\_user` | Generate random alphanumeric string: `openssl rand -base64 24 \| tr -dc 'a-zA-Z0-9' \| head -c 20` |

| `DJANGO\_SECRET\_KEY` | Cryptographic secret for session cookies and tokens | Generate with: `python3 -c "import secrets; print(secrets.token\_urlsafe(50))"` |

| `ADMIN\_URL\_PREFIX` | Obfuscated admin path (e.g. `control-hub/` or `admin/`) | Choose custom path to deter bot scanners from hitting `admin/`. |

| `DEFAULT\_OWNER\_ID` | Staff login ID for primary Owner (e.g. `Salman`) | Custom business owner username for POS login. |

| `DEFAULT\_OWNER\_PASSWORD` | Strong password for primary Owner | Create a secure, strong alphanumeric password. |

| `YOUR\_EMAIL` | Admin email address for Let's Encrypt certificate renewal alerts | Your active business/admin email. |

[!IMPORTANT]

***Use Letters & Numbers Only for `DB\_PASSWORD`**: Special characters like `@`, `:`, `/`, or `%` in passwords can cause URI-encoding syntax errors with database connection strings. Stick to alphanumeric characters (`A-Z`, `a-z`, `0-9`).*

2. Findings from the Codebase Inspection

During inspection of the `Wondersale-os` repository, several crucial architecture and configuration findings were discovered:

2.1 Complete Environment Variable Reference (`backend/.env`)

| Variable Name | Required? | Default in Code | Purpose & Recommended Production Setting |

|---|---|---|

| `DJANGO\_ENV` | Yes | `development` | Set to `production`. |

| `DJANGO\_DEBUG` | Yes | `True` | ****Must be `False`**** in production. Never deploy with `True`. |

| `DJANGO\_SECRET\_KEY` | Yes | ***Hardcoded dev key*** | Must be set to a cryptographically secure random string. Django raises an `ImproperlyConfigured` exception if missing when `DEBUG=False`. |

| `ALLOWED\_HOSTS` | Yes | `wonder.sale,api.wonder.sale` | Comma-separated list of hostnames allowed to serve requests. Set to `YOUR\_DOMAIN,www.YOUR\_DOMAIN,YOUR\_SERVER\_IP,127.0.0.1,localhost`. |

| `ADMIN\_URL\_PREFIX` | No | `admin/` | Prefix for Django Admin. Trailing slash is automatically enforced. E.g. `control-hub/`. |

| `POSTGRES\_DB` | Yes | *(Triggers SQLite fallback if omitted)* | Set to `wondersale`. Presence of this variable tells Django to use PostgreSQL instead of SQLite3! |

| `POSTGRES\_USER` | Yes | `postgres` | Set to `wondersale\_user`. |

| `POSTGRES\_PASSWORD` | Yes | `postgres` | Set to your strong alphanumeric `DB\_PASSWORD`. |

| `POSTGRES\_HOST` | Yes | `localhost` | Set to `localhost` or `127.0.0.1`. |

| `POSTGRES\_PORT` | No | `5432` | Standard PostgreSQL port `5432`. |

| `POSTGRES\_CONNECT\_TIMEOUT` | No | `10` | Database connection timeout in seconds. |

| `POSTGRES\_SSLMODE` | No | `require` (when DEBUG=False) | Local PostgreSQL on same host does not use SSL sockets. ****Must be set to `prefer` or `disable`**** when connecting to local unix socket/localhost. |

| `CORS\_ALLOWED\_ORIGINS` | Yes | `https://wonder.sale,...` | Comma-separated list of origins allowed for Cross-Origin requests. Set to `https://YOUR\_DOMAIN,https://www.YOUR\_DOMAIN`. |

| `CSRF\_TRUSTED\_ORIGINS` | Yes | `https://wonder.sale,...` | Comma-separated list of origins trusted for CSRF token POST validation. Set to `https://YOUR\_DOMAIN,https://www.YOUR\_DOMAIN`. |

| `SECURE\_SSL\_REDIRECT` | No | `True` (when DEBUG=False) | Set to `False` in `.env` initially until Certbot SSL is active, or let Nginx handle the HTTPS redirect. |

| `SECURE\_HSTS\_SECONDS` | No | `31536000` | HTTP Strict Transport Security duration (1 year). |

| `DJANGO\_LOG\_LEVEL` | No | `INFO` | File & console log verbosity level (`INFO`, `WARNING`, `ERROR`). |

| `WEBP\_QUALITY` | No | `80` | Compression quality for automatic product image conversion. |

| `DEFAULT\_OWNER\_ID` | No | `Salman` | Staff ID bootstrap for the first Owner account created on initial launch. |

| `DEFAULT\_OWNER\_PASSWORD` | No | `7869186388` | Default bootstrap password. ****MUST CHANGE IN PRODUCTION!**** |

| `DEFAULT\_STORE\_NAME` | No | `Bhopal Plaza` | Name of the primary retail branch store auto-created on initial launch. |

| `DEFAULT\_STORE\_PHONE` | No | `9755004996` | Store contact number. |

| `DEFAULT\_STORE\_GST` | No | `23ANGPK5446D2Z8` | Store GST / Tax Identification Number. |

| `GEMINI\_API\_KEYS` | Optional | *Empty* | Comma-separated Google Gemini API keys used for receipt OCR. |

| `META\_WHATSAPP\_PHONE\_NUMBER\_ID` | Optional | *Empty* | Meta Cloud API Phone Number ID for sending digital receipts. |

| `META\_WHATSAPP\_ACCESS\_TOKEN` | Optional | *Empty* | Meta Cloud API Permanent System User token. |

| `META\_WHATSAPP\_BUSINESS\_ACCOUNT\_ID` | Optional | *Empty* | WhatsApp Business Account (WABA) ID. |

| `META\_WHATSAPP\_RETURN\_TEMPLATE\_NAME` | Optional | `return\_receipt` | Template name approved in Meta Business Manager. |

| `MAX\_DAILY\_WHATSAPP\_MESSAGES` | Optional | `500` / `5000` | Safety ceiling for automated WhatsApp notifications. |

| `ORDER\_RESEND\_COOLDOWN\_SECONDS` | Optional | `60` | Cooldown period between duplicate invoice sends. |

| `MAX\_MESSAGES\_PER\_PHONE\_PER\_HOUR` | Optional | `10` | Anti-spam limit per customer phone number. |

| `META\_API\_TIMEOUT\_SECONDS` | Optional | `10` | HTTP timeout when contacting Graph API. |

2.2 Critical Risks & Gotchas Identified

1. ****`unicorn` Missing from `requirements.txt`**:**

The repository `requirements.txt` contains 82 dependencies, but `unicorn` itself is not listed. If you only run `pip install -r requirements.txt`, Unicorn will fail to execute with `unicorn: command not found`. Our step installs `unicorn` explicitly.

2. ****`reportlab` Missing from `requirements.txt`**:**

`backend/inventory/whatsapp\_service.py` directly imports `reportlab` to render tax invoice PDF receipts on the fly. Although present in local development, it is missing from `requirements.txt`. Without it, sending WhatsApp invoices crashes with `ModuleNotFoundError: No module named 'reportlab'`. Our guide installs `reportlab` explicitly.

3. ****Database Driver `psycopg` vs `psycopg2`**:**

`backend/requirements.txt` includes `psycpg==3.3.5` and `psycpg-binary==3.3.5` (Psycpg 3). Django 5.2 natively supports Psycpg 3. You do **not** need the older `psycpg2-binary`.

4. **PostgreSQL SSL Mode Gotcha** (`POSTGRES\_SSLMODE`)**:

`wondersale\_core/settings.py` sets `sslmode = 'require'` by default when `DEBUG=False` unless `POSTGRES\_SSLMODE` is explicitly provided in `.env`. Connecting to a local PostgreSQL instance (`localhost`) via TCP or socket will fail if SSL is forced but PostgreSQL does not have SSL certificates configured. You must set `POSTGRES\_SSLMODE=prefer` or `disable` in `.env`.

5. **Initial Owner & Store Bootstrap**:

`staff/services/auth.py` has automatic bootstrapping built into the login/view endpoints (`ensure\_default\_roles\_and\_owner()` and `ensure\_default\_store()`). When the database is fresh, it creates the store `Bhopal Plaza` and user `Salman` (with fallback password `7869186388` if not provided in `.env`). To prevent insecure default credentials from ever being written, configure `DEFAULT\_OWNER\_ID` and `DEFAULT\_OWNER\_PASSWORD` in `.env` *before* initial login!

6. **Frontend API URL Architecture**:

In `frontend/src/api.js`, all endpoints use relative paths: `/api/inventory`, `/api/staff`, `/api/stakeholders`. No hardcoded `localhost:8000` or `127.0.0.1` exists in the frontend source code. This means the production Vite build will seamlessly call the domain hosting it, and Nginx will reverse-proxy `/api/` to Gunicorn.

7. **Barcodes & Font Dependencies**:

`backend/inventory/services.py` uses `OCR-B.ttf` located in `backend/inventory/fonts/OCR-B.ttf` (present in repo). If that file is ever missing, it falls back to `/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf`. We install `fonts-dejavu-core` via `apt` as a safe fallback.

8. **Automated Directory Creation**:

`settings.py` automatically runs `MEDIA\_ROOT.mkdir(parents=True, exist\_ok=True)` and `LOGS\_DIR.mkdir(exist\_ok=True)`. However, permissions must belong to `jason:www-data` so both the Gunicorn worker and Nginx can read and write media files and logs.

2.3 Items to Check Manually on Server

- **DNS Propagation**: Before requesting a Let's Encrypt SSL certificate, ensure your domain's DNS A record points to `YOUR\_SERVER\_IP` using `dig +short YOUR\_DOMAIN` or `nslookup YOUR\_DOMAIN`.
- **VPS Memory (RAM)**: Running `npm run build` with Vite + React 19 on small 1 GB / 2 GB RAM VPS instances can trigger Linux OOM Killer (`Killed`). We include swap creation commands in Section 3.2.
- **WhatsApp Webhook Configuration**: If you enable Meta WhatsApp notifications, configure your webhook callback URL and verify token inside Meta App Dashboard.

3. Step-by-Step Production Deployment

Part A: Verify Starting Point

Run these commands to confirm your existing baseline configuration on your Ubuntu 24.04 VPS.

Step A1: Verify user identity

```
whoami
```

- **What it does**: Verifies that your active shell session is logged in as `jason`.
- **Expected result**: `jason` (NOT `root`).
- **If it fails**: If it returns `root`, switch to user jason: `su - jason`.

Step A2: Verify sudo privileges

```
sudo whoami
```

- **What it does**: Checks that user `jason` can run administrative commands via `sudo`.
- **Expected result**: `root`.

- **If it fails:** If prompted that `jason` is not in the sudoers file, log in as root and run `usermod -aG sudo jason`, then log back in as `jason`.

Step A3: Verify UFW firewall status

```
sudo ufw status verbose
```

- **What it does:** Checks active firewall rules. OpenSSH (port 22) must be allowed.
- **Expected result:** `Status: active` with port `22/tcp` or `OpenSSH` allowed in the rule list.

[!WARNING]

DO NOT OPEN PORT 8000 OR 5432: Old tutorials instruct running `sudo ufw allow 8000` to test Django runserver externally. Never do this! Port 8000 (Gunicorn) and 5432 (PostgreSQL) must remain strictly internal on `127.0.0.1`.

Part B: System Packages, Memory Swap & Node.js 22 LTS

Step B1: Refresh package index and upgrade packages

```
sudo apt update && sudo apt upgrade -y
```

- **What it does:** Synchronizes repository package indexes and installs latest security patches.
- **Expected result:** `All packages are up to date.`

Step B2: Install build tools, database packages, and system libraries

```
sudo apt install -y git curl wget build-essential libpq-dev \
python3-venv python3-dev python3-pip nginx fonts-dejavu-core
```

- **What it does:** Installs essential compilers, PostgreSQL headers (`libpq-dev`), Python virtual environment support (`python3-venv`), Nginx web server, and fallback TrueType fonts for PDF/barcode rendering.
- **Expected result:** All packages installed without error.

Step B3: Setup 2 GB Swap Space (Prevents OOM Crashes During Build)

```
if [ $(swapon --show | wc -l) -le 1 ]; then
  sudo fallocate -l 2G /swapfile
  sudo chmod 600 /swapfile
  sudo mkswap /swapfile
  sudo swapon /swapfile
  echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
fi
free -h
```

- **What it does:** Adds a 2 GB swap file if none exists. React 19 + Vite builds can spike memory usage over 1.5 GB; swap ensures the build does not fail with `Killed`.
- **Expected result:** `Swap:` row shows `2.0Gi` total.

Step B4: Install Node.js 22 LTS via NodeSource

Ubuntu 24.04 apt repositories bundle Node.js v18 by default. React 19 and modern Vite tooling require Node 20 or 22.

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
```

- **What it does:** Registers official NodeSource repository for Node.js 22 and installs `nodejs` and `npm`.
- **Expected result:** Installation finishes successfully.

Step B5: Verify Node and NPM versions

```
node -v && npm -v
```

- **Expected result:** Node shows `v22.x.x` and npm shows `10.x.x`.

Part C: Clone Repository into `/var/www/wondersale`

Ubuntu 24.04 sets strict `750` permissions on user home folders (`/home/jason`). If a web app is hosted in `/home/jason/`, Nginx worker processes running under `www-data` cannot traverse the directory, resulting in stubborn `403 Forbidden` errors. Placing the

application in `/var/www/` adheres to standard FHS (Filesystem Hierarchy Standard).

Step C1: Create directory and set ownership to `json`

```
sudo mkdir -p /var/www/wondersale
sudo chown -R json:json /var/www/wondersale
```

- **What it does:** Creates target root directory and grants ownership to user `json`.
- **Expected result:** Directory created with ownership `json:json`.

Step C2: Clone the repository

```
git clone https://github.com/AmmarFaizulHasan/Wondersale-os.git /var/www/wondersale
```

(If prompted for authentication or using your own fork, use your repository URL).

- **What it does:** Downloads the Wondersale project codebase into `/var/www/wondersale`.
- **Expected result:** `Cloning into '/var/www/wondersale'... done.`

Step C3: Verify folder structure

```
ls -la /var/www/wondersale
```

- **Expected result:** You should see `backend`, `frontend`, `README.md`, `brand-kit`, etc.

Part D: PostgreSQL 16 Installation & Database Setup

Step D1: Install PostgreSQL 16

```
sudo apt install -y postgresql postgresql-contrib
```

- **What it does:** Installs PostgreSQL server and extensions. On Ubuntu 24.04, this installs PostgreSQL 16.
- **Expected result:** PostgreSQL service starts automatically.

Step D2: Verify PostgreSQL service status

```
sudo systemctl status postgresql --no-pager
```

- **Expected result:** `active (running)`.

Step D3: Create database and user

Replace `DB_PASSWORD` with your chosen alphanumeric password.

```
sudo -u postgres psql -c "CREATE USER wondersale_user WITH PASSWORD 'DB_PASSWORD';"
sudo -u postgres psql -c "CREATE DATABASE wondersale OWNER wondersale_user;"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE wondersale TO wondersale_user;"
sudo -u postgres psql -c "ALTER USER wondersale_user CREATEDB;"
```

- **What it does:** Creates PostgreSQL user `wondersale_user`, creates database `wondersale`, assigns ownership, and grants full privileges.
- **Expected result:** Each command outputs `CREATE ROLE`, `CREATE DATABASE`, `GRANT`, `ALTER ROLE`.

Step D4: Test database connection

```
PGPASSWORD='DB_PASSWORD' psql -h localhost -U wondersale_user -d wondersale -c "SELECT version();" 
```

- **What it does:** Verifies that `wondersale_user` can authenticate over localhost using the password.
- **Expected result:** Prints PostgreSQL 16 version string.
- **If it fails:** If `FATAL: password authentication failed`, re-run Step D3 and make sure your password contains only alphanumeric letters and numbers.

Part E: Python Virtual Environment & Backend Dependencies

[!IMPORTANT]

PEP 668 on Ubuntu 24.04: Ubuntu 24.04 marks system Python as an "externally managed environment". Running `sudo pip3 install ...` will fail. You must create an isolated virtual environment using `python3 -m venv`. Never use `--break-system-packages`.

Step E1: Create Python virtual environment

```
cd /var/www/wondersale/backend
python3 -m venv venv
```

- **What it does:** Creates a clean, self-contained Python 3.12 environment in `/var/www/wondersale/backend/venv`.
- **Expected result:** `venv/` directory created with `bin/python` and `bin/pip`.

Step E2: Upgrade pip and setuptools inside venv

```
/var/www/wondersale/backend/venv/bin/pip install --upgrade pip setuptools wheel
```

- **What it does:** Ensures modern packaging utilities inside the virtual environment.
- **Expected result:** `Successfully installed pip-...`.

Step E3: Install requirements plus Gunicorn and ReportLab

As discovered in Section 2, `gunicorn` and `reportlab` are missing from `requirements.txt`. Install all requirements together:

```
/var/www/wondersale/backend/venv/bin/pip install -r requirements.txt
/var/www/wondersale/backend/venv/bin/pip install gunicorn reportlab
```

- **What it does:** Installs all 82 project dependencies (including Django 5.2, Psycopg 3, Argon2, Pillow, DRF) plus Gunicorn WSGI server and ReportLab invoice generator.
- **Expected result:** `Successfully installed Django-5.2... gunicorn-... reportlab-...`.

Step E4: Verify critical modules inside virtual environment

```
/var/www/wondersale/backend/venv/bin/python -c "
import django, psycopg, gunicorn, reportlab, PIL
print('Django:', django.__version__)
print('Psycopg (Postgres driver):', psycopg.__version__)
print('Gunicorn:', gunicorn.__version__)
print('ReportLab:', reportlab.__version__)
print('Pillow:', PIL.__version__)
"
```

- **Expected result:** All version strings print without any `ImportError` or `ModuleNotFoundError`.

Part F: Production Environment Configuration (`.env`)

Step F1: Generate a strong Django Secret Key

Run this command to generate your production secret key:

```
python3 -c "import secrets; print(secrets.token_urlsafe(50))"
```

Copy the printed output string.

Step F2: Create `/var/www/wondersale/backend/.env`

Create the file using `nano`:

```
nano /var/www/wondersale/backend/.env
```

Paste the following configuration, replacing `YOUR_SECRET_KEY`, `YOUR_DOMAIN`, `YOUR_SERVER_IP`, `DB_PASSWORD`, and initial owner credentials:

```
# =====
# WONDERSALE SYSTEM - PRODUCTION ENVIRONMENT CONFIGURATION
# =====

DJANGO_ENV=production
DJANGO_DEBUG=False
DJANGO_SECRET_KEY=YOUR_SECRET_KEY

# Allowed Hosts (include your domain, www subdomain, server IP, and localhost)
ALLOWED_HOSTS=YOUR_DOMAIN,www.YOUR_DOMAIN,YOUR_SERVER_IP,127.0.0.1,localhost

# Obfuscated Admin URL Prefix (must end with /)
ADMIN_URL_PREFIX=admin/

# Initial Bootstrap Owner & Store (created automatically on first launch)
```

```

DEFAULT_OWNER_ID=Salman
DEFAULT_OWNER_PASSWORD=YOUR_STRONG_OWNER_PASSWORD
DEFAULT_STORE_NAME=Bhopal Plaza
DEFAULT_STORE_PHONE=9755004996
DEFAULT_STORE_GST=23ANGPK5446D2Z8

# PostgreSQL Database Configuration
POSTGRES_DB=wondersale
POSTGRES_USER=wondersale_user
POSTGRES_PASSWORD=DB_PASSWORD
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
POSTGRES_CONNECT_TIMEOUT=10
POSTGRES_SSLMODE=prefer

# CORS & CSRF Trusted Origins (HTTPS URLs only)
CORS_ALLOWED_ORIGINS=https://YOUR_DOMAIN,https://www.YOUR_DOMAIN
CSRF_TRUSTED_ORIGINS=https://YOUR_DOMAIN,https://www.YOUR_DOMAIN

# SSL & Security Redirects (Let Nginx handle initial HTTP->HTTPS redirect)
SECURE_SSL_REDIRECT=False
SECURE_HSTS_SECONDS=31536000

# Logging
DJANGO_LOG_LEVEL=INFO

# Optional: Google Gemini AI API Keys for Bill OCR (comma-separated)
# GEMINI_API_KEYS=your_key_1,your_key_2

# Optional: Meta WhatsApp Cloud API for Digital Receipts
# META_WHATSAPP_PHONE_NUMBER_ID=
# META_WHATSAPP_ACCESS_TOKEN=
# META_WHATSAPP_BUSINESS_ACCOUNT_ID=
# META_WHATSAPP_RETURN_TEMPLATE_NAME=return_receipt
MAX_DAILY_WHATSAPP_MESSAGES=5000
ORDER_RESEND_COOLDOWN_SECONDS=60
MAX_MESSAGES_PER_PHONE_PER_HOUR=10
META_API_TIMEOUT_SECONDS=10

```

Press `Ctrl + O`, then `Enter` to save, and `Ctrl + X` to exit `nano`.

Step F3: Lock down `.env` file permissions

```
chmod 600 /var/www/wondersale/backend/.env
```

- **What it does:** Restricts read and write permissions exclusively to user `jason`. Prevents any unauthorized user on the system from inspecting database credentials or the Django secret key.

- **Expected result:** Permissions show `-rw-----`.

Part G: Migrations, Superuser, Static Files & Directory Permissions

Step G1: Apply Database Migrations

```
cd /var/www/wondersale/backend
/var/www/wondersale/backend/venv/bin/python manage.py migrate
```

- **What it does:** Connects to PostgreSQL database `wondersale` and creates all database tables for inventory, accounting, staff, stakeholders, auth, and sessions.

- **Expected result:** Output lists `Applying <app>.0001\_initial... OK` across all apps.

- **If it fails:** If `django.db.utils.OperationalError: connection to server at "127.0.0.1" failed`, check that PostgreSQL is running (`sudo systemctl status postgresql`) and verify credentials in `.env`.

Step G2: Create Django Superuser (Interactive)

```
/var/www/wondersale/backend/venv/bin/python manage.py createsuperuser
```

- **What it does:** Creates an administrative Django account for accessing the Django Admin interface (`/admin/`).

- **Input needed:**

- Username (e.g. `admin` or your personal name)
- Email address
- Password (minimum 12 characters as enforced by `settings.py`)
- **Expected result:** `Superuser created successfully.`

Step G3: Collect Static Files

```
/var/www/wondersale/backend/venv/bin/python manage.py collectstatic --noinput
```

- **What it does:** Gathers all Django admin CSS, JavaScript, and rest_framework static files into `backend/staticfiles/` so Nginx can serve them directly.
- **Expected result:** `xxx static files copied to '/var/www/wondersale/backend/staticfiles'`.

Step G4: Create directories and set group ownership to `www-data`

Both Gunicorn (running as user `jason`, group `www-data`) and Nginx (running as user `www-data`) need access to static, media, and log directories.

```
mkdir -p /var/www/wondersale/backend/media
mkdir -p /var/www/wondersale/backend/logs
mkdir -p /var/www/wondersale/backend/staticfiles

sudo chown -R jason:www-data /var/www/wondersale/backend/media
sudo chown -R jason:www-data /var/www/wondersale/backend/logs
sudo chown -R jason:www-data /var/www/wondersale/backend/staticfiles

chmod -R 775 /var/www/wondersale/backend/media
chmod -R 775 /var/www/wondersale/backend/logs
chmod -R 755 /var/www/wondersale/backend/staticfiles
```

- **What it does:** Ensures `media/` (where product photos and receipts are stored) and `logs/` (where `wondersale.log` is written) are writable by both `jason` and `www-data`.
- **Expected result:** Directories created with `rwxrwxr-x` permissions.

Part H: Manual Gunicorn Test with `curl`

Before setting up a systemd background service, test Gunicorn by hand in the foreground to verify there are no hidden Python runtime errors.

Step H1: Start Gunicorn manually in the terminal

```
cd /var/www/wondersale/backend
/var/www/wondersale/backend/venv/bin/gunicorn \
  --workers 2 \
  --bind 127.0.0.1:8000 \
  --timeout 60 \
  wondersale_core.wsgi:application
```

- **What it does:** Boots Gunicorn directly in your shell.
- **Expected result:** Output displays:

...

[INFO] Starting gunicorn ...

[INFO] Listening at: http://127.0.0.1:8000

[INFO] Using worker: sync

[INFO] Booting worker with pid: ...

...

Step H2: Test connection with curl (Open a Second Terminal or SSH Session)

Open a second terminal window, connect via SSH as `jason`, and run:

```
curl -I -H "Host: YOUR_DOMAIN" http://127.0.0.1:8000/api/inventory/stores/
```

- **What it does:** Sends an HTTP request directly to Gunicorn on local port 8000, passing `Host: YOUR_DOMAIN` so Django's `ALLOWED_HOSTS` validation succeeds.
- **Expected result:** HTTP status `HTTP/1.1 200 OK` (or `401 Unauthorized` / `301 Moved Permanently`).
- **If it fails with 400 Bad Request:** Your `Host:` header did not match `ALLOWED_HOSTS` in `.env`. Ensure `YOUR_DOMAIN` is inside `ALLOWED_HOSTS`.

Step H3: Stop manual Gunicorn

Switch back to your first terminal window and press `Ctrl + C` to stop the foreground Gunicorn process.

Part I: Frontend Production Build

The frontend is a React 19 Single Page Application. In production, Vite compiles all JSX, icons, and components into static HTML, JavaScript, and CSS bundles in `frontend/dist`.

Step I1: Verify API client configuration

As confirmed during repo inspection, `frontend/src/api.js` uses relative paths (`/api/inventory`, `/api/staff`). It does not hardcode any localhost URLs. Vite connects to whatever domain serves the page.

Step I2: Install frontend npm dependencies

```
cd /var/www/wondersale/frontend
npm install
```

- **What it does:** Downloads and installs all Node dependencies (`react`, `react-dom`, `@dnd-kit`, `framer-motion`, `lucide-react`, `jspdf`, `vite`).
- **Expected result:** `added xxx packages, and audited xxx packages in ...s`.

Step I3: Build production assets

```
npm run build
```

- **What it does:** Runs `vite build` to generate minified production bundles in `/var/www/wondersale/frontend/dist`.
- **Expected result:** Output displays:

...

vite v8.x.x building for production...

✓ built in ...s

dist/index.html ... kB

dist/assets/index-...js ... kB

dist/assets/index-...css ... kB

...

- **If it fails with `Killed`:** The VPS ran out of RAM! Ensure you completed Step B3 (2 GB swap file).

Step I4: Set permissions on frontend dist directory

```
sudo chown -R jason:www-data /var/www/wondersale/frontend/dist
sudo chmod -R 755 /var/www/wondersale/frontend/dist
```

- **What it does:** Allows Nginx worker processes (`www-data`) to read the compiled HTML, CSS, and JS files.

Part J: Configure Systemd Service (`wondersale.service`)

Create a systemd unit file to manage the Gunicorn WSGI process as a background daemon that automatically restarts on failures and boots on server restart.

Step J1: Create `/etc/systemd/system/wondersale.service`

```
sudo nano /etc/systemd/system/wondersale.service
```

Paste the following configuration:

```

[Unit]
Description=Gunicorn daemon for Wondersale Retail Backend
After=network.target postgresql.service
Wants=postgresql.service

[Service]
User=jason
Group=www-data
WorkingDirectory=/var/www/wondersale/backend
EnvironmentFile=/var/www/wondersale/backend/.env
ExecStart=/var/www/wondersale/backend/venv/bin/gunicorn \
    --workers 3 \
    --bind 127.0.0.1:8000 \
    --timeout 120 \
    --access-logfile /var/www/wondersale/backend/logs/gunicorn_access.log \
    --error-logfile /var/www/wondersale/backend/logs/gunicorn_error.log \
    wondersale_core.wsgi:application
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

```

Press `Ctrl + O`, `Enter`, and `Ctrl + X` to save.

[!NOTE]

***Why `User=jason` and `Group=www-data`? Running as `jason` ensures files created by Django retain permissions accessible to your deploy user, while `Group=www-data` allows Nginx to read shared files such as uploaded media.*

Step J2: Reload systemd daemon

```
sudo systemctl daemon-reload
```

- **What it does:** Notifies systemd of the new service file.

Step J3: Start and enable `wondersale.service`

```
sudo systemctl start wondersale
sudo systemctl enable wondersale
```

- **What it does:** Starts Gunicorn immediately and configures it to launch on system boot.
- **Expected result:** Created symlink /etc/systemd/system/multi-user.target.wants/wondersale.service...

Step J4: Verify service status

```
sudo systemctl status wondersale --no-pager
```

- **Expected result:** Active: active (running) with 3 Gunicorn worker processes.

Part K: Configure Nginx & Firewall

Nginx acts as the front-line web server:

- Serves React SPA files (`/var/www/wondersale/frontend/dist`) with fallback routing (`try\_files \$uri \$uri /index.html`).
- Proxies `/api/` and `/admin/` requests to Gunicorn on `127.0.0.1:8000`.
- Serves `/static/` and `/media/` files directly.
- Implements `client\_max\_body\_size 20M` to support high-resolution product photos.

Step K1: Remove the default Nginx placeholder site

```
sudo rm -f /etc/nginx/sites-enabled/default
```

- **What it does:** Disables the standard "Welcome to nginx" default page.

Step K2: Create Nginx site configuration

```
sudo nano /etc/nginx/sites-available/wondersale
```

Paste the following configuration (replace `YOUR\_DOMAIN` with your actual domain name):

```

server {
    listen 80;
    listen [::]:80;
    server_name YOUR_DOMAIN www.YOUR_DOMAIN;

    client_max_body_size 20M;

    # Frontend Single Page Application (React / Vite)
    root /var/www/wondersale/frontend/dist;
    index index.html;

    # Handle React SPA client-side routing
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Proxy Django API requests to Gunicorn
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 60s;
        proxy_read_timeout 120s;
    }

    # Proxy Django Admin interface
    location /admin/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 60s;
        proxy_read_timeout 120s;
    }

    # Django Collected Static Files
    location /static/ {
        alias /var/www/wondersale/backend/staticfiles/;
        expires 30d;
        add_header Cache-Control "public, max-age=2592000, immutable";
    }

    # Django Uploaded Media Files (Product Images, Receipts)
    location /media/ {
        alias /var/www/wondersale/backend/media/;
        expires 7d;
        add_header Cache-Control "public, max-age=604800";
    }
}

```

Press `Ctrl + O`, `Enter`, and `Ctrl + X` to save.

Step K3: Enable the site by creating a symbolic link

```
sudo ln -sf /etc/nginx/sites-available/wondersale /etc/nginx/sites-enabled/
```

- **What it does:** Links the site into `sites-enabled` so Nginx will load it.

Step K4: Test Nginx configuration syntax

```
sudo nginx -t
```

- **What it does:** Validates configuration syntax for errors before restarting.

- **Expected result:**

...

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok

nginx: configuration file /etc/nginx/nginx.conf test is successful

...

- **If it fails:** Check the output for line numbers indicating typos or mismatched braces.

Step K5: Allow Nginx through UFW Firewall

```
sudo ufw allow 'Nginx Full'
sudo ufw status
```

- **What it does:** Opens port 80 (HTTP) and port 443 (HTTPS) in UFW while leaving port 22 (SSH) open and keeping internal ports (8000, 5432) blocked.

- **Expected result:** `Nginx Full` shows `ALLOW` in the rule list.

Step K6: Restart Nginx

```
sudo systemctl restart nginx
```

- **What it does:** Reloads Nginx with the new Wondersale reverse-proxy configuration.

Part L: HTTPS with Let's Encrypt Certbot & Django HTTPS Settings

Step L1: Install Certbot and the Nginx plugin

```
sudo apt install -y certbot python3-certbot-nginx
```

- **What it does:** Installs the Let's Encrypt automated SSL certificate issuance tool.

Step L2: Obtain and install SSL Certificate

Run Certbot (replace `YOUR\_DOMAIN` and `YOUR\_EMAIL`):

```
sudo certbot --nginx -d YOUR_DOMAIN -d www.YOUR_DOMAIN --non-interactive --agree-tos -m YOUR_EMAIL --redirect
```

- **What it does:** Validates domain ownership via ACME HTTP-01 challenge, issues an SSL certificate, modifies `/etc/nginx/sites-available/wondersale` to configure SSL ciphers, and configures automatic HTTP->HTTPS redirection.

- **Expected result:**

...

Successfully received certificate.

Certificate is saved at: /etc/letsencrypt/live/YOUR_DOMAIN/fullchain.pem

...

Congratulations! You have successfully enabled HTTPS on https://YOUR_DOMAIN

...

- **If it fails:** Ensure your domain's DNS A record points directly to `YOUR\_SERVER\_IP` and has finished propagating.

Step L3: Test automatic renewal

```
sudo certbot renew --dry-run
```

- **What it does:** Simulates certificate renewal to verify that the systemd timer (`certbot.timer`) can renew certificates without manual intervention.

- **Expected result:** `Congratulations, all simulated renewals succeeded.`

Step L4: Enable Django SSL Redirect in `.env`

Now that HTTPS is active, update Django settings to enforce SSL redirects and secure cookies.

Open `.env`:

```
nano /var/www/wondersale/backend/.env
```

Change:

```
SECURE_SSL_REDIRECT=True
```

Save with `Ctrl + O`, `Enter`, and exit with `Ctrl + X`.

Restart Gunicorn to apply:

```
sudo systemctl restart wondersale
```

Part M: Final Verification Checklist

Perform these tests in order to confirm full operational readiness:

1. **Browser Test:**

- Open `https://YOUR\_DOMAIN` in your browser.
- Verify the lock icon appears in the address bar (HTTPS).
- Verify the Wondersale glassmorphic login screen loads cleanly without errors.

2. **Staff Login Bootstrap Test:**

- Log in using your configured `DEFAULT\_OWNER\_ID` (e.g. `Salman`) and `DEFAULT\_OWNER\_PASSWORD`.
- Verify login succeeds and redirects to the dashboard.
- Verify the initial store (e.g. `Bhopal Plaza`) is automatically selected.

3. **Django Admin Interface:**

- Navigate to `https://YOUR\_DOMAIN/admin/` (or your custom `ADMIN\_URL\_PREFIX`).
- Log in with the superuser created in Step G2.
- Verify Django Admin loads with full CSS styling (proving `/static/` is correctly served by Nginx).

4. **Media Upload Test:**

- Navigate to the Inventory module.
- Add a product and upload a sample image.
- Verify the image displays properly in the table (proving `/media/` is correctly served by Nginx).

5. **Server Reboot Test (Guarantees Persistence):**

```
``bash
```

```
sudo reboot
```

```
``
```

Wait 60 seconds, reconnect via SSH as `jason`, and verify:

```
``bash
```

```
sudo systemctl status wondersale --no-pager
```

```
sudo systemctl status nginx --no-pager
```

```
sudo systemctl status postgresql --no-pager
```

```
``
```

All three services must show `Active: active (running)`.

4. Maintenance & Updating the Site

When you push new code to your Git repository, use this standard sequence to pull changes, migrate the database, and rebuild assets.

4.1 Update Script Workflow

Run these commands as user `jason`:

```
cd /var/www/wondersale
```

```
# 1. Fetch latest changes
git pull origin main
```

```
# 2. Update Python dependencies (if requirements changed)
/var/www/wondersale/backend/venv/bin/pip install -r backend/requirements.txt
/var/www/wondersale/backend/venv/bin/pip install gunicorn reportlab

# 3. Apply database migrations
/var/www/wondersale/backend/venv/bin/python backend/manage.py migrate

# 4. Collect any updated static files
/var/www/wondersale/backend/venv/bin/python backend/manage.py collectstatic --noinput

# 5. Restart backend service
sudo systemctl restart wondersale

# 6. Rebuild frontend assets
cd /var/www/wondersale/frontend
npm install
npm run build

# 7. Reload Nginx cache
sudo systemctl reload nginx
```

4.2 Handling Local Edits vs `git pull`

If you made emergency edits directly on the server, `git pull` may fail with merge conflicts.

- **View local modifications:**

```
```bash
git status
git diff
```
```

- **Stash local edits before pulling:**

```
```bash
git stash
git pull origin main
git stash pop
```
```

- **Discard local edits and force-match GitHub:**

```
```bash
git reset --hard origin/main
```
```

5. Automated Backups & Server Hardening

5.1 Automated Nightly Backup Script

Create a robust backup script that dumps the PostgreSQL database, compresses uploaded media assets, and purges backups older than 14 days.

Step 1: Create backup directory and script

```
sudo mkdir -p /var/backups/wondersale
sudo chown -R jason:jason /var/backups/wondersale
nano /home/jason/backup_wondersale.sh
```

Paste the following script:

```
#!/bin/bash
set -e

TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
```

```

BACKUP_DIR="/var/backups/wondersale"
DB_NAME="wondersale"
DB_USER="wondersale_user"
MEDIA_DIR="/var/www/wondersale/backend/media"

# Read password from .env file
DB_PASS=$(grep -E '^POSTGRES_PASSWORD=' /var/www/wondersale/backend/.env | cut -d '=' -f2 | tr -d '"')

# 1. Dump PostgreSQL Database (compressed custom format)
PGPASSWORD="$DB_PASS" pg_dump -h 127.0.0.1 -U "$DB_USER" -Fc "$DB_NAME" > "$BACKUP_DIR/db_${TIMESTAMP}.dump"

# 2. Archive uploaded media
tar -czf "$BACKUP_DIR/media_${TIMESTAMP}.tar.gz" -C "$MEDIA_DIR" .

# 3. Keep only the last 14 days of backups
find "$BACKUP_DIR" -type f -mtime +14 -delete

echo "[${TIMESTAMP}] Backup completed successfully." >>> "$BACKUP_DIR/backup.log"

```

Save and exit `nano`.

Step 2: Make executable and test

```

chmod +x /home/jason/backup_wondersale.sh
/home/jason/backup_wondersale.sh
ls -lh /var/backups/wondersale

```

- **Expected result:** You will see `db\_YYYYMMDD\_HHMMSS.dump` and `media\_YYYYMMDD\_HHMMSS.tar.gz`.

Step 3: Schedule Nightly Cron Job at 2:00 AM

```
crontab -e
```

Add this line at the bottom:

```
0 2 * * * /home/jason/backup_wondersale.sh >>/dev/null 2>&1
```

5.2 Server Hardening Best Practices

1. Configure SSH Key Authentication & Disable Password Login

[!WARNING]

*****CRITICAL: NEVER CLOSE YOUR CURRENT TERMINAL BEFORE TESTING!*****

If you disable password login without verifying that your SSH key works in a separate terminal window, you could lock yourself out of your server permanently!

- **On your local computer:** Copy your public key to the server:

```
``bash
```

```
ssh-copy-id jason@YOUR_SERVER_IP
```

```
``
```

- **Open a NEW terminal on your local computer** and verify you can log in without entering a password:

```
``bash
```

```
ssh jason@YOUR_SERVER_IP
```

```
``
```

- **Once confirmed working**, edit SSH configuration on the server:

```
``bash
```

```
sudo nano /etc/ssh/sshd_config.d/50-cloud-init.conf
```

```
``
```

**(Or `/etc/ssh/sshd\_config` if that file does not exist)*.*

Ensure these directives are set:

```
``ini
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
---
```

- **Restart the SSH service** (On Ubuntu 24.04 the service name is `ssh`):

```
``bash
sudo systemctl restart ssh
---
```

2. Install and Configure Fail2ban (Brute-Force Shield)

```
sudo apt install -y fail2ban
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
```

- **What it does:** Monitors authentication logs and automatically bans IP addresses that show malicious brute-force attempts.

3. Enable Unattended Security Upgrades

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

- **What it does:** Automatically installs critical Linux kernel and security patches in the background.

6. Comprehensive Troubleshooting Matrix

| Symptom | Most Likely Cause | Exact Fix |

|---|---|---|

| **502 Bad Gateway** | Unicorn service is stopped, crashed, or failing to bind to `127.0.0.1:8000`. | Check status: `sudo systemctl status wondersale`
Inspect logs: `sudo journalctl -u wondersale -n 50 --no-pager`
Look for Python tracebacks (e.g. missing import, syntax error). |

| **400 Bad Request** | Request `Host` header does not match `ALLOWED\_HOSTS` in `backend/.env`. | Open `.env` and add your exact domain and server IP to `ALLOWED\_HOSTS`:
`ALLOWED\_HOSTS=YOUR\_DOMAIN,www.YOUR\_DOMAIN,YOUR\_SERVER\_IP,127.0.0.1`
Then run: `sudo systemctl restart wondersale` |

| **403 Forbidden on Web Pages** | Nginx cannot read `/var/www/wondersale/frontend/dist` or parent folders. | Check permissions: `sudo chmod 755 /var/www` && `sudo chmod -R 755 /var/www/wondersale/frontend/dist`
Ensure files are owned by `jason:www-data`. |

| **404 Not Found on Page Refresh** | Nginx is missing `try\_files \$uri \$uri/ /index.html;` in `location /`. | Verify `location /` in `/etc/nginx/sites-available/wondersale` has `try\_files \$uri \$uri/ /index.html;`. Then test: `sudo nginx -t` && `sudo systemctl reload nginx`. |

| **Blank Page / Network Errors** | Frontend calling wrong API URL or API request blocked by CORS. | Ensure Nginx config contains `location /api/ { proxy\_pass http://127.0.0.1:8000; ... }`.
Check browser Developer Tools Console (F12) for specific failing URL. |

| **Django Admin Missing CSS / Styling** | Static files were not collected or Nginx static alias is incorrect. | Run:
`/var/www/wondersale/backend/venv/bin/python /var/www/wondersale/backend/manage.py collectstatic --noinput`
Verify `/etc/nginx/sites-available/wondersale` has `location /static/ { alias /var/www/wondersale/backend/staticfiles/; }`. |

| **CSRF Verification Failed (403)** | `CSRF\_TRUSTED\_ORIGINS` in `.env` is missing `https://YOUR\_DOMAIN`. | In `.env`:
`CSRF\_TRUSTED\_ORIGINS=https://YOUR\_DOMAIN,https://www.YOUR\_DOMAIN`
Note: Must include `https://` prefix! Then run: `sudo systemctl restart wondersale`. |

| **Too Many Redirects (ERR_TOO_MANY_REDIRECTS)** | Loop between Nginx SSL redirect and Django `SECURE\_SSL\_REDIRECT`. | In `.env`, set `SECURE\_SSL\_REDIRECT=False` and ensure Nginx sends `proxy\_set\_header`

X-Forwarded-Proto \$scheme;` so Django recognizes HTTPS. |

| **413 Request Entity Too Large** | Uploading product images larger than Nginx's default 1MB limit. | Add `client\_max\_body\_size 20M;` inside `server { ... }` in `/etc/nginx/sites-available/wondersale`.
Reload Nginx: `sudo systemctl reload nginx`. |

| **Permission Denied on `logs/` or `media/`** | Gunicorn cannot write to `backend/logs/wondersale.log` or save images in `media/`. | Run: `sudo chown -R jason:www-data /var/www/wondersale/backend/logs /var/www/wondersale/backend/media`
`sudo chmod -R 775 /var/www/wondersale/backend/logs /var/www/wondersale/backend/media`. |

| **Database Authentication Failed (FATAL: password authentication failed)** | Incorrect `POSTGRES\_PASSWORD` in `.env` or password contains special characters. | Reset user password in PostgreSQL:
`sudo -u postgres psql -c "ALTER USER wondersale\_user WITH PASSWORD 'NEW\_ALPHA\_PASSWORD';"`
Update `.env` with the same password and restart service. |

| **npm run build` Failed: "Killed"** | VPS RAM exhausted by Node.js process during Vite build. | Add 2 GB swap space as explained in Section 3.2, Step B3: <br>`sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile`. |

| **wondersale.service` Fails to Start** | Gunicorn binary missing or incorrect working directory. | Test manual invocation: `var/www/wondersale/backend/venv/bin/gunicorn --version` <br>If missing, run `var/www/wondersale/backend/venv/bin/pip install gunicorn`. <br>View full logs: `sudo journalctl -u wondersale -e`. |

Appendix A: Differences from Legacy Guides (CodeWithHarry & Repo README)

If you are accustomed to older Ubuntu 20.04 tutorials (such as CodeWithHarry) or the brief generic production section in the repo README, take note of these critical architectural differences:

| Topic | Old / Legacy Tutorial Approach | Why This Guide Uses a Different Method |

---|---|---

| **Python Virtual Environment** | `sudo pip3 install virtualenv` | **Fails on Ubuntu 24.04** due to PEP 668 ("externally managed environment"). This guide uses Ubuntu 24.04's native `python3 -m venv`. |

| **Project Location** | `/home/jason/wondersale` | In Ubuntu 24.04, `/home/jason` has restrictive `750` permissions. Nginx runs as `www-data` and cannot access the home directory, triggering `403 Forbidden`. This guide deploys to `/var/www/wondersale`. |

| **Gunicorn Architecture** | Gunicorn Socket Activation (`.socket` + `.service`) | Socket activation introduces complex debugging when systemd socket permissions desynchronize. A single standalone `wondersale.service` bound to `127.0.0.1:8000` is vastly more reliable and simpler to troubleshoot. |

| **Port 8000 & Firewall** | `sudo ufw allow 8000` to test Django runserver | Exposing Django development server or Gunicorn directly over the internet is a major security vulnerability. Gunicorn is kept internal on `127.0.0.1:8000`; Nginx handles all public traffic. |

| **Database Engine** | SQLite3 fallback | SQLite lacks multi-user concurrency for retail POS cashiers. This guide uses PostgreSQL 16 with optimized connection pooling (`CONN\_MAX\_AGE=600`). |

| **Node.js Installation** | `sudo apt install nodejs` (installs v18) | Ubuntu 24.04 apt repos bundle Node 18, which is too old for React 19 and Vite 8. This guide installs Node.js 22 LTS via the official NodeSource script. |

| **Missing Dependencies** | Assumed `requirements.txt` has everything | The repo omitted `gunicorn` and `reportlab` from `requirements.txt`. This guide explicitly installs them to prevent runtime crashes. |

| **Service User** | Repo README used `User=www-data` | Using `User=jason` and `Group=www-data` allows the deployment user to run migrations, inspect files, and manage logs without requiring `sudo` for every command. |

Appendix B: Command Cheat Sheet

Keep these commands handy for day-to-day server administration:

Service Management

```
# Restart Backend Unicorn
sudo systemctl restart wondersale

# Check Backend Status
sudo systemctl status wondersale --no-pager

# Restart Nginx Web Server
sudo systemctl restart nginx

# Reload Nginx without dropping connections
sudo systemctl reload nginx

# Test Nginx configuration syntax
sudo nginx -t

# Restart PostgreSQL Database
sudo systemctl restart postgresql
```

Log Inspection

```
# Live tail of Unicorn systemd service logs
sudo journalctl -u wondersale -f

# View last 100 lines of Django application log
tail -n 100 -f /var/www/wondersale/backend/logs/wondersale.log

# Live tail of Nginx error logs
sudo tail -f /var/log/nginx/error.log

# Live tail of Nginx access logs
sudo tail -f /var/log/nginx/access.log
```

PostgreSQL Direct Access

```
# Open interactive PostgreSQL shell as wondersale_user
PGPASSWORD='DB_PASSWORD' psql -h 127.0.0.1 -U wondersale_user -d wondersale

# View list of database tables
\dt

# Exit psql prompt
\q
```

Firewall & SSL

```
# Check UFW firewall status
sudo ufw status verbose

# Test SSL certificate auto-renewal
sudo certbot renew --dry-run
```